TRAVELSYNC

Intelligent Travel Planning & Information Platform

PROJECT DOCUMENTATION

Data Structures and Algorithms (DSA) Project | Review-1 (CO1–CO3)
B.Tech – Computer Science

Subject Code	25CS2103E
Team Members	1. Y. Keerthika 2. N. Jyothirmayee 3. M. Mallika
Guide	M. Syamala Mam

Abstract

TravelSync is a web application for travel planning and travel information. It lets users explore and search destinations, view nearby places, compare destinations, plan trips and receive preference-based recommendations. Behind these features sit fifteen Data Structures and Algorithms (DSA) engines written in Java: string-search algorithms, text-comparison and sequence-alignment algorithms, dynamic-programming techniques and a network-flow activity allocator.

The frontend is built with React and Vite, and the backend with Java 21 and Spring Boot, exposing REST APIs. The application also uses external services (Open-Meteo, Wikipedia, OSRM routing and OpenStreetMap Overpass) to obtain live travel information. The algorithm implementations are verified by an automated Java test suite (14 tests passed in the latest checked report, with focused tests since added for the activity allocator).

Keywords: Travel planning, string matching, edit distance, sequence alignment, bitmask dynamic programming, network flow, Edmonds–Karp, Spring Boot, React.

1. Introduction

1.1 Overview

Travel information is large, spread across many sources and often unstructured. A traveller may misspell a destination name, want to compare two destination descriptions, need to visit several places in an efficient order, or want recommendations that fit personal preferences. Each of these is a well-defined computational problem that can be solved with a suitable data structure or algorithm.
TravelSync was developed as a DSA project to show that classical algorithms can be integrated into a practical, user-facing application. Instead of presenting algorithms in isolation, the project connects each one to a real travel feature.

1.2 Problem Statement

Modern travel platforms handle large amounts of destination, booking and descriptive information. Searching, comparing and organizing this information efficiently needs suitable data structures and algorithms. The main challenges addressed are:

- Finding relevant travel information quickly in text.
- Handling misspelled destination queries.
- Comparing destination descriptions to find shared or similar text.
- Arranging multi-stop routes in an efficient order.
- Choosing activities and destinations that fit user preferences.

TravelSync applies string matching, text comparison and dynamic programming to these tasks within one web application.

1.3 Objectives

1. Build a modular web application with a separate frontend and backend.
2. Apply string-search and string-matching algorithms to travel data.
3. Compare travel descriptions using text-comparison and alignment algorithms.
4. Optimize travel plans using dynamic programming and allocate travelers to activities using network flow.
5. Integrate external travel-information and routing services.
6. Test the algorithm implementations with an automated test suite and analyse their complexity.

2. Scope and Technology

2.1 Implemented Scope

- A React + Vite web interface that calls REST APIs.
- A Spring Boot backend with controllers, services, models, external API clients and DSA engine classes.
- Destination exploration and search, nearby places, destination comparison, trip planning, recommendations and algorithm insights.
- Integration with Open-Meteo, Wikipedia, OSRM and OpenStreetMap Overpass.
- Sample in-memory booking records with booking-code search.
- An activity allocator based on Network Flow (Edmonds–Karp), exposed through a REST API and the trip-planner UI.
- Fifteen DSA engines and an automated Java test suite.

2.2 Current Limitations

- There is no database; booking records are sample in-memory data.
- The application is not a complete hotel-booking system. There is no Hotel model and no hotel-search feature.
- No performance benchmarks on real-world datasets have been carried out.

2.3 Technology Stack

Table 2.1: Technology stack

Layer	Technology
Frontend	React with Vite
Backend	Java 21 with Spring Boot
Communication	REST APIs exchanging JSON between frontend and backend
Algorithms	Custom DSA engine classes written in Java
External services	Open-Meteo, Wikipedia, OSRM, OpenStreetMap Overpass
Testing	Automated Java test suite (14 tests passed in the latest checked report; focused allocator tests added)

3. System Architecture

TravelSync follows a layered, modular architecture. The React/Vite frontend calls a Spring Boot REST API. Requests pass through controllers to services, which use the DSA engines and the external API clients, and the results are returned to the frontend as JSON.

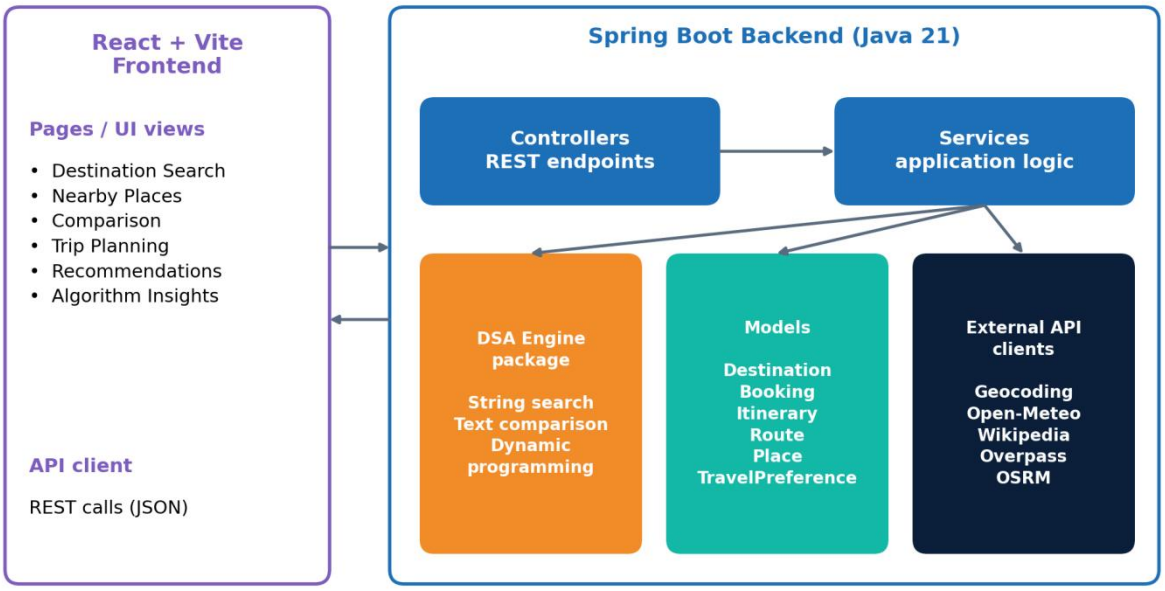


Figure 3.1: TravelSync system architecture

3.1 Architecture Layers

Table 3.1: Components and responsibilities

Component	Responsibility
Frontend	Pages for destination search, nearby places, destination comparison, trip planning, recommendations and algorithm insights, plus an API client that makes REST calls.
Controllers	Receive HTTP requests from the frontend and delegate to services.
Services	Hold the application logic. They call the DSA engines and external API clients and assemble the response.
DSA engine package	The implemented algorithms, kept independent of the web layer so that they can be tested on their own.
Models	Destination, Booking, Itinerary, Route, Place and TravelPreference.
External API clients	Clients for geocoding, weather, Wikipedia information, nearby places and routing.

4. Workflow, Features and Services

4.1 System Workflow

1. The user opens the web application and selects a feature.
2. The frontend sends a REST API request to the backend.
3. The controller passes the request to the relevant service.
4. The service calls the appropriate DSA engine and/or external API client.
5. The backend returns a JSON result.
6. The frontend displays the result, and the user can select another feature.

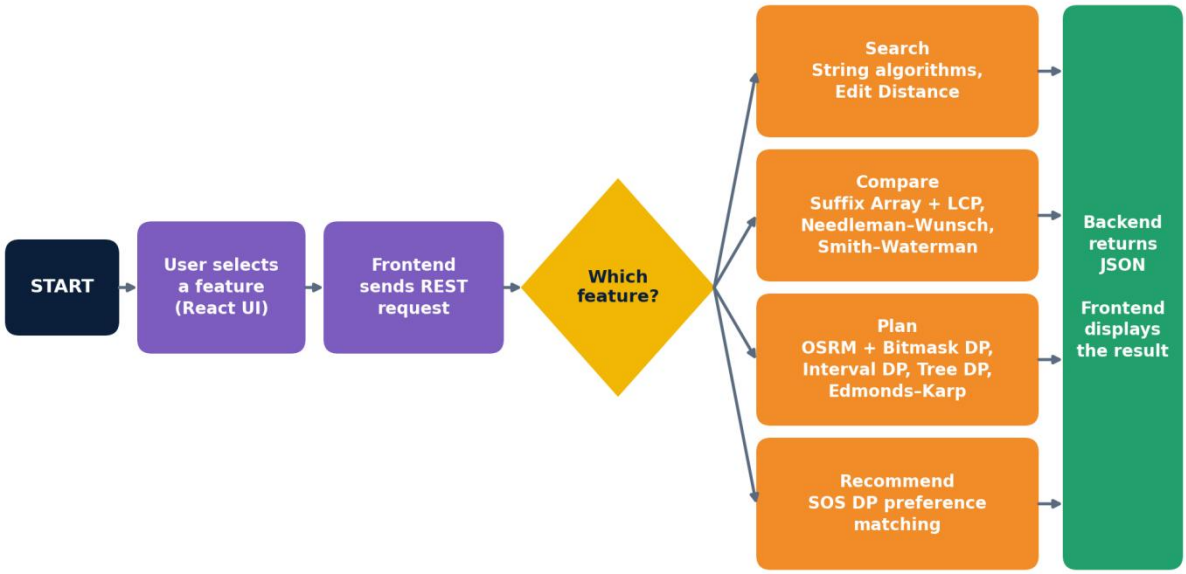


Figure 4.1: System workflow

4.2 Application Features

Table 4.1: User-facing features

Feature	Description
Destination exploration and search	Find destinations and view information about them, with suggestions when a query is misspelled.
Nearby places	Discover places around a destination using OpenStreetMap Overpass data.
Destination comparison	Compare destinations and their descriptions.
Trip planning	Order multi-stop routes, schedule activities and optimize hierarchical travel plans.

Feature	Description
Activity allocation	Assign travelers to compatible activities within seat capacities using Network Flow (Edmonds–Karp) in the planner UI.
Recommendations	Rank destinations by the user's travel preferences.
Algorithm insights	A view that explains the algorithms used by the application.
Booking-code lookup	Search sample in-memory booking records by booking code.

4.3 External Services

Table 4.2: External services

Service	Purpose
Open-Meteo	Weather information.
Wikipedia	Descriptive information about destinations.
OSRM	Routing; supplies the route matrix used for multi-destination optimization.
OpenStreetMap Overpass	Nearby places.

The backend calls these services through dedicated client classes, so the algorithm code stays separate from network code.

4.4 Frontend–Backend Interaction

The frontend does not run the algorithms itself. Each feature sends a REST request to the backend, the backend runs the relevant service and returns a JSON response, and the frontend renders the result. This keeps the user interface thin and keeps all algorithmic logic in the Java backend, where it can be tested independently.

- **Request:** user input such as a search query, selected destinations or travel preferences.
- **Processing:** controller → service → DSA engine and/or external API client.
- **Response:** a JSON result that the React pages display.

4.5 Design Principles

- **Separation of concerns:** controllers, services, models, external clients and algorithm engines are separate components.
- **Algorithm independence:** the DSA engines do not depend on the web layer, so they can be tested on their own.
- **External service isolation:** network calls are confined to dedicated client classes.

5. Data Model

The backend uses the following model classes to carry data between the controllers, services and algorithm engines.

Table 5.1: Model classes

Model	Description
Destination	A travel destination with descriptive information and category tags.
Booking	A sample in-memory booking record identified by a booking code.
Itinerary	An ordered plan of destinations or activities.
Route	Route information between destinations, with distances or durations obtained from routing data.
Place	A nearby place of interest.
TravelPreference	The user's travel preferences, represented as categories and bit masks.

5.2 Where Each Model Is Used

Table 5.2: Model usage

Model	Typical use
Destination	Destination search, typo suggestions, description comparison and recommendations.
Booking	Booking-code search over sample in-memory records.
Itinerary	Trip planning, including ordered routes and scheduled activities.
Route	Route costs between destinations, used by multi-destination optimization.
Place	Nearby-places results.
TravelPreference	Preference masks used for recommendations.

5.3 Data Handling Notes

- Booking records are sample in-memory data; no database is used.
- Travel preferences and destination categories are encoded as bit masks so that SOS DP can compare them quickly.
- Route data is obtained from OSRM, nearby places from OpenStreetMap Overpass, weather from Open-Meteo and destination information from Wikipedia.
- Data moves through the layers as: frontend JSON request → controller → service → model objects and algorithm engine → JSON response.

6. DSA Implementation

Fifteen algorithm engines are implemented. They fall into four groups: string search and matching, text comparison and sequence alignment, dynamic programming for planning, and network flow for activity allocation.

6.1 String Search and Matching

Table 6.1: String search algorithms

Algorithm	How it works	Use in TravelSync
Naive String Search	Compares the pattern against every position of the text.	Baseline string matching
KMP	Builds a failure (prefix) table so the pattern is matched without re-reading text characters.	Pattern searching
Z-Function	For each position, finds the longest substring matching a prefix of the string; applied to pattern + text it finds occurrences.	Pattern searching
Rabin–Karp	Compares rolling hash values of the pattern and of text windows, and verifies on a hash match.	Booking-code search
Aho–Corasick	Builds a trie of many keywords with failure links and scans the text once.	Multiple keyword search

6.2 Text Comparison and Sequence Alignment

Table 6.2: Text comparison algorithms

Algorithm	How it works	Use in TravelSync
Edit Distance	Minimum number of insertions, deletions and substitutions to turn one string into another (Wagner–Fischer).	Destination typo suggestions
Suffix Array	Sorted array of all suffixes of a string.	Longest common substring (with LCP)
LCP (Kasai)	Longest-common-prefix array of adjacent suffixes, computed in linear time.	Longest common substring (with Suffix Array)
Needleman–Wunsch	Global alignment of two complete sequences using a scoring scheme.	Comparing whole destination descriptions
Smith–Waterman	Local alignment that finds the best-matching region of two sequences.	Strongest shared portion of two descriptions

6.3 Dynamic Programming for Planning

Table 6.3: Dynamic-programming algorithms

Algorithm	How it works	Use in TravelSync
Bitmask DP	State = (set of visited destinations, current destination) stored as a bit mask.	Multi-destination route ordering
Interval DP	Sorts activities and uses a predecessor lookup to choose a non-overlapping set.	Activity scheduling
Tree DP	Combines the best budget splits across the children of each node in a tree of choices.	Hierarchical travel-plan optimization under a budget
SOS DP	Computes aggregate values over all subsets of an 8-bit preference mask.	Preference-based recommendations

Edit Distance, Needleman–Wunsch and Smith–Waterman are also dynamic-programming techniques.

6.4 Network Flow

Table 6.4: Network-flow algorithm

Algorithm	How it works	Use in TravelSync
Edmonds–Karp	Repeatedly finds a shortest augmenting path with BFS in the residual graph and pushes flow along it; residual edges let earlier choices be revised.	Assigning travelers to compatible activities within seat capacities

6.5 Why These Techniques Were Chosen

Table 6.5: Problem-to-technique rationale

Problem	Technique	Reason
Typo suggestions	Edit Distance	Counts the edits separating two strings.
Shared text	Suffix Array + LCP	Finds the longest common substring efficiently.
Description alignment	Needleman–Wunsch, Smith–Waterman	Scoring handles inserted, changed or missing words.
Route ordering	Bitmask DP	Tracks visited subsets exactly for a small number of stops.
Preferences	SOS DP	Aggregates over all subsets of a preference mask.
Seat allocation	Edmonds–Karp	Maximizes assignments and revises choices through residual edges.

7. Key Workflows

7.1 Multi-Destination Itinerary Optimization

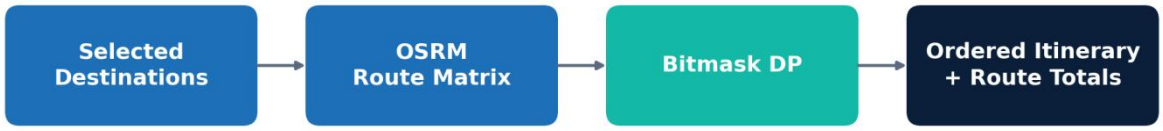


Figure 7.1: Itinerary optimization workflow

1. The user selects the destinations to visit.
2. The backend requests a route matrix for those destinations from OSRM.
3. Bitmask DP explores combinations of visited destinations and chooses an ordering with the lowest total route cost.
4. The ordered itinerary and its route totals are returned to the frontend.

Route costs come from OSRM; the algorithm selects the optimized ordering. The time complexity is $O(n^2 \cdot 2^n)$, where n is the number of destinations, so the method suits a small number of stops.

7.2 Travel Preference Matching with SOS DP

There are eight preference categories, each assigned one bit of an 8-bit mask. A destination's categories and a user's preferences are both represented as masks, and SOS DP computes subset aggregates over all $2^8 = 256$ masks to rank recommendations.

Table 7.1: Preference categories and bit masks

Category	Bit position	Mask
Beach	0	00000001
Historical	1	00000010
Adventure	2	00000100
Nature	3	00001000
Food	4	00010000
Nightlife	5	00100000
Relaxation	6	01000000
Shopping	7	10000000

Illustrative example: a destination tagged Beach + Nightlife + Relaxation has the mask 00000001 + 00100000 + 01000000 = 01100001 (bits 0, 5 and 6 set; 1 + 32 + 64 = 97).

7.3 Activity Allocation with Network Flow

The activity allocator assigns each traveler to at most one compatible activity while respecting each activity's seat capacity. It is solved as a maximum-flow problem with the Edmonds–Karp algorithm.

Table 7.2: Flow-network model

Element	Meaning
Source → traveler	Capacity 1: each traveler can be assigned at most once.
Traveler → activity	Capacity 1: only where the traveler is compatible with the activity.
Activity → sink	Capacity = the activity's seats.
Maximum flow	The maximum number of travelers that can be assigned.

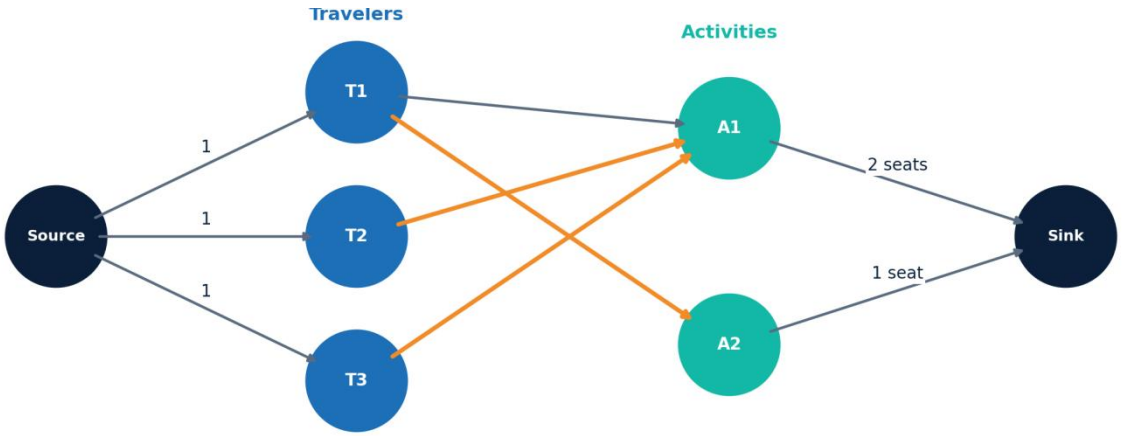


Figure 7.2: Illustrative flow network (orange edges show one maximum assignment)

1. Build the network from the travelers, activities, compatibilities and seat capacities.
2. Edmonds–Karp finds shortest augmenting paths using BFS and pushes flow along them.
3. Residual edges let the algorithm undo and revise an earlier assignment when that frees a seat for another traveler.
4. When no augmenting path remains, the flow is maximum; the assignments are read from the edges carrying flow.

The solver is part of the backend DSA engine package, is exposed through a REST API and is used by the trip-planner UI. Its time complexity is $O(V \cdot E^2)$, where V is the number of vertices (source, travelers, activities, sink) and E is the number of edges.

8. Testing

The repository contains an automated Java test suite of **14 tests**. The latest checked test report showed **14 tests passed and 0 failures**. Focused tests for the Network Flow activity allocator were added afterwards.
The tests cover the algorithms listed below.

Table 8.1: Algorithm tests covered

No.	Algorithm	No.	Algorithm
1	Naive Search	8	LCP
2	KMP	9	Needleman–Wunsch
3	Z-Function	10	Smith–Waterman
4	Rabin–Karp	11	Bitmask DP
5	Aho–Corasick	12	Interval DP
6	Edit Distance	13	Tree DP
7	Suffix Array	14	SOS DP

Note: this result is taken from the latest checked report. No tests were run while preparing this document.

8.1 Testing Approach

- Each algorithm engine is tested on its own, independent of the web layer.
- Tests use small, deterministic inputs with known expected outputs.

8.2 Coverage by Algorithm Group

Table 8.2: Tests by group

Group	Algorithms covered	Tests
String search and matching	Naive, KMP, Z-Function, Rabin–Karp, Aho–Corasick	5
Text comparison and alignment	Edit Distance, Suffix Array, LCP, Needleman–Wunsch, Smith–Waterman	5
Dynamic programming	Bitmask DP, Interval DP, Tree DP, SOS DP	4
Network flow	Edmonds–Karp activity allocator	Focused tests added after the checked report

9. Complexity Analysis

Table 9.1: Time complexity of the implemented algorithms

Algorithm	Time Complexity	Variables / Notes
Naive Search	O(nm)	n = text length, m = pattern length
KMP	O(n + m)	
Z-Function	O(n + m)	
Rabin–Karp	Average O(n + m); worst case O(nm)	Worst case with many hash collisions
Aho–Corasick	O(total pattern length + text length + matches)	
Edit Distance	O(nm)	n, m = lengths of the two strings
Suffix Array	$O(n \log^2 n)$	Prefix doubling with comparison sort
LCP (Kasai)	O(n)	n = string length
Needleman–Wunsch	O(nm)	n, m = sequence lengths
Smith–Waterman	O(nm)	n, m = sequence lengths
Bitmask DP	$O(n^2 \cdot 2^n)$	n = number of destinations
Interval DP (scheduling)	O(n log n)	n = activities; sorting and predecessor lookup
Tree DP	$O(N \cdot B^2)$ time; $O(N \cdot B)$ space	N = tree nodes, B = budget
Edmonds–Karp (Network Flow)	$O(V \cdot E^2)$	V = vertices, E = edges of the flow network
SOS DP	$O(k \cdot 2^k)$	k = number of preference bits (8)

No benchmark timings are reported in this document.

9.1 Notes on the Results

- KMP, Z-Function and Aho–Corasick run in time linear in the text and pattern sizes, which improves on the O(nm) naive baseline.
- Bitmask DP is exponential in the number of destinations, so it suits a small number of stops.
- Edmonds–Karp runs in $O(V \cdot E^2)$ regardless of the capacity values.

10. Conclusion and Future Scope

10.1 Conclusion

TravelSync shows how advanced Data Structures and Algorithms can be integrated into a practical travel-planning web application. It combines a React web interface, external travel-data services and a Java Spring Boot backend with DSA-powered features: string search, typo suggestions, description comparison, route ordering, activity scheduling, hierarchical plan optimization, preference-based recommendations and activity allocation using network flow. The modular design keeps the algorithms separate from the web layer, which makes them easy to test and extend.

10.2 Future Scope

- Persistent database storage in place of in-memory booking records.
- Broader live datasets.
- More robust personalization of recommendations.
- Deployment of the application.
- Additional real-world performance evaluation of the algorithms.

10.3 Summary of Contributions

Table 10.1: Summary

Area	Contribution
Web application	React + Vite frontend and Java 21 Spring Boot backend connected through REST APIs.
Algorithms	Fifteen DSA engines: string search, text comparison, dynamic programming and network flow.
Integration	Open-Meteo, Wikipedia, OSRM and OpenStreetMap Overpass.
Planning features	Route ordering, activity scheduling, hierarchical plans, recommendations and activity allocation.
Testing	Automated Java test suite covering the algorithm engines.

10.4 Learning Outcomes

- Designing dynamic-programming solutions for routing, scheduling, hierarchical optimization and preference matching.
- Modelling an allocation problem as a flow network and solving it with Edmonds–Karp.

References

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press.
2. D. Gusfield, Algorithms on Strings, Trees, and Sequences, Cambridge University Press.
3. Open-Meteo API documentation.
4. Wikipedia API documentation.
5. OSRM (Open Source Routing Machine) documentation.
6. OpenStreetMap Overpass API documentation.
7. J. Edmonds and R. M. Karp, Theoretical improvements in algorithmic efficiency for network flow problems, Journal of the ACM, 1972.
8. Spring Boot, React and Vite official documentation.

Appendix A: Application Screenshots

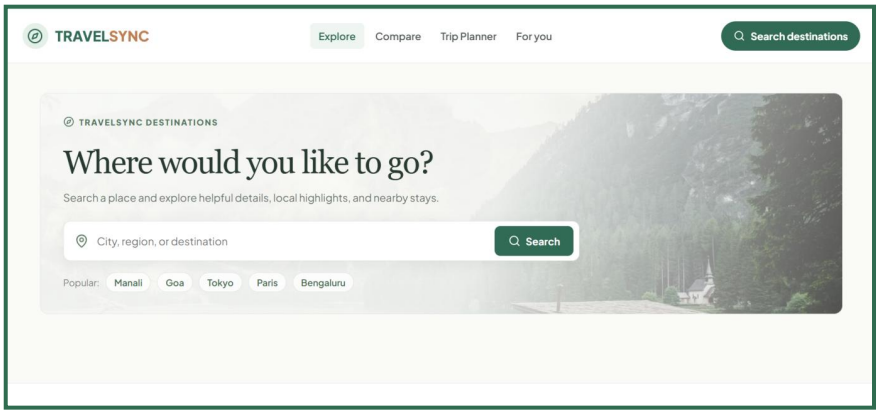


Figure A.1: Home page: destination search

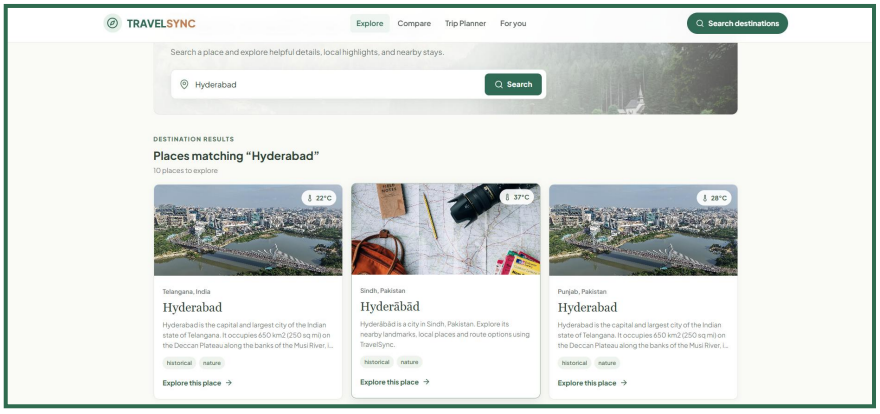


Figure A.2: Search results for "Hyderabad"

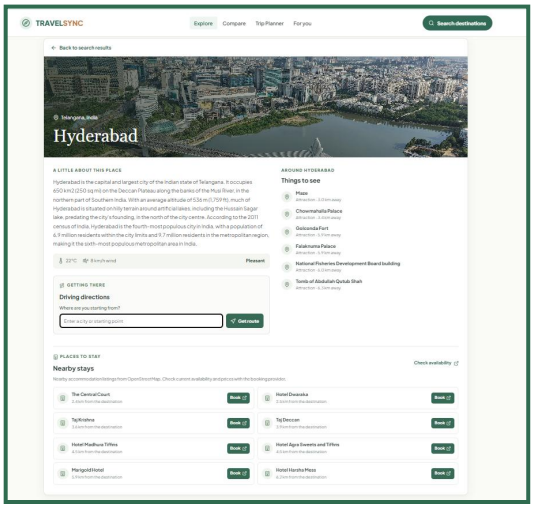


Figure A.3: Destination page with things to do and nearby stays

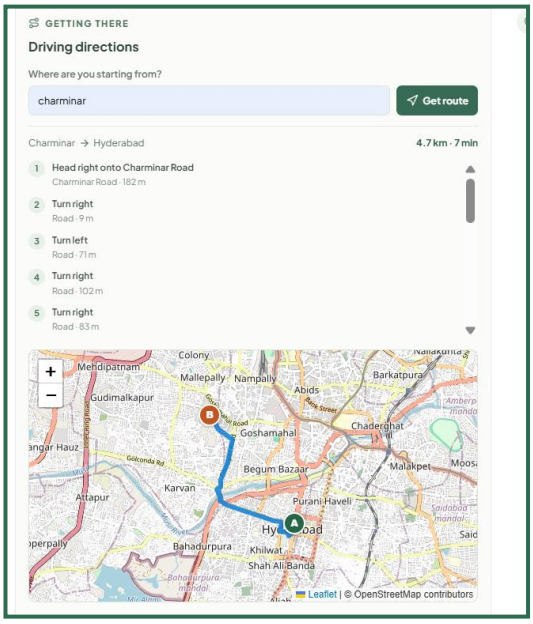


Figure A.4: Driving directions with route map

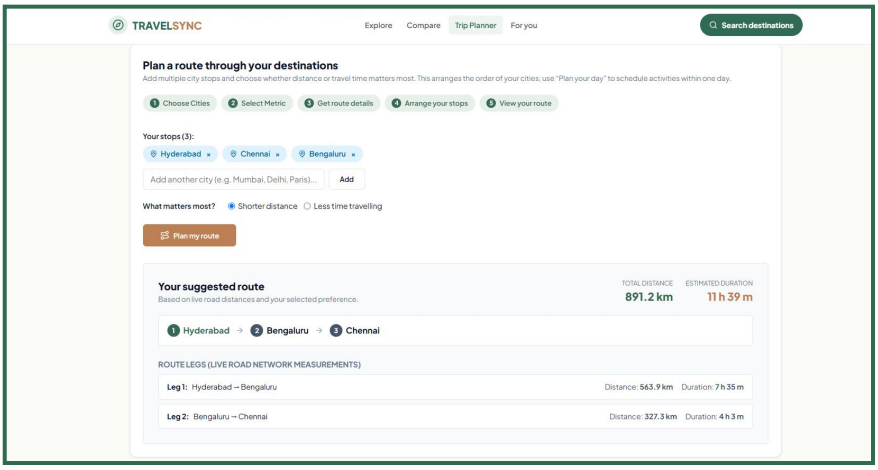


Figure A.5: Route planner: suggested multi-city route

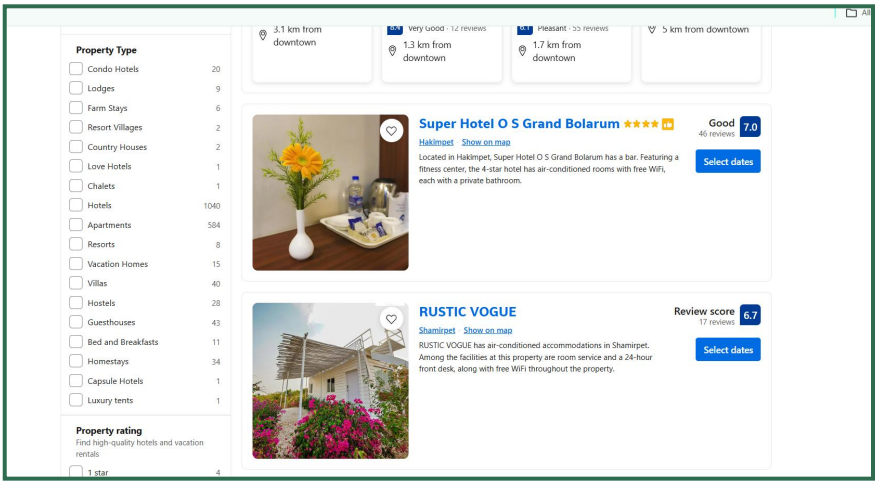


Figure A.6: Nearby stays: accommodation listing

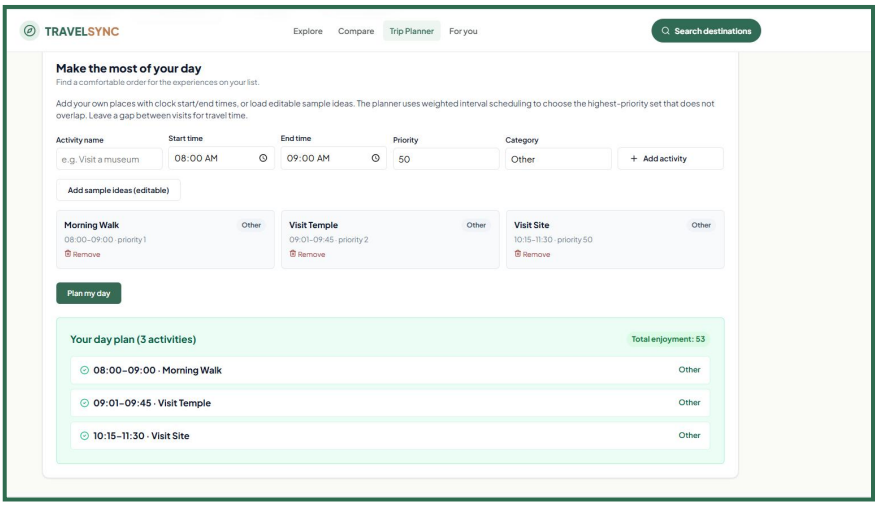


Figure A.7: Day planner: optimized daily itinerary

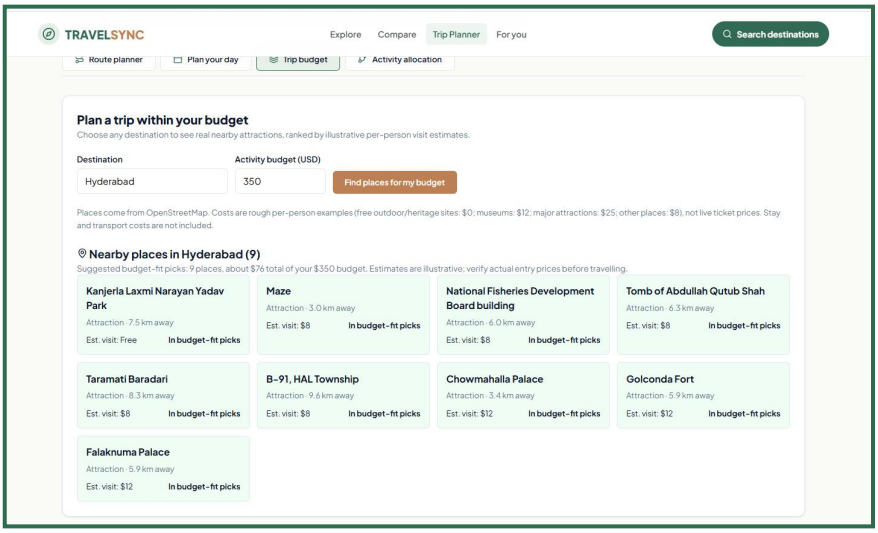


Figure A.8: Budget planner: places within budget

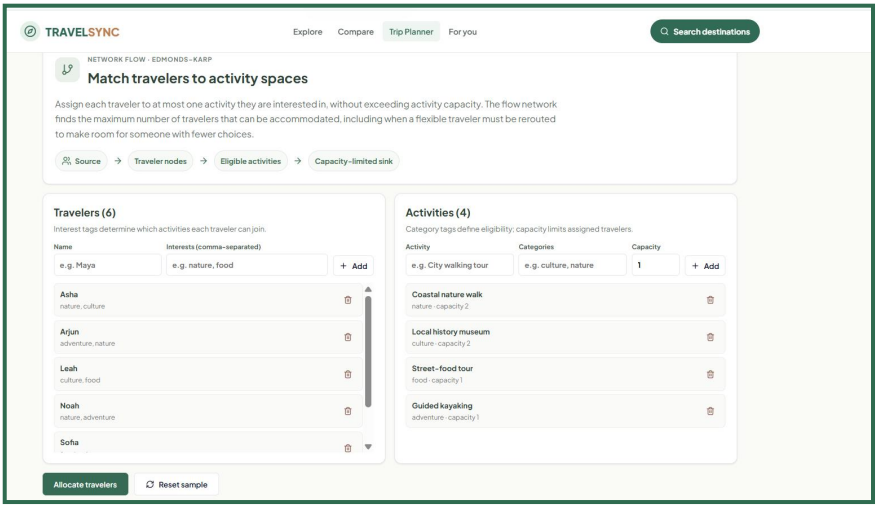


Figure A.9: Match travelers to activity spaces

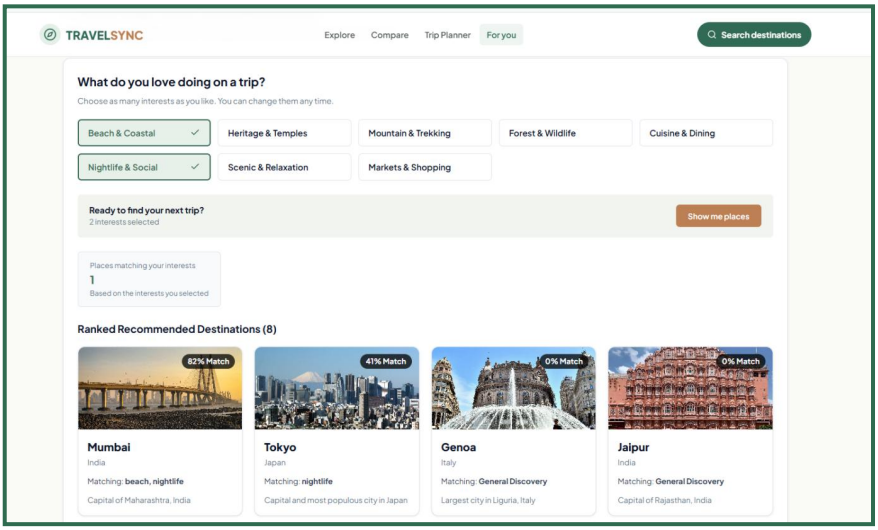


Figure A.10: For You: personalized destination recommendations